

1. ¿Es necesario almacenar toda la información para luego procesarla?

Si el enunciado no pide explícitamente que la información se almacene, entonces no debe almacenarse. Almacenar tiene costos extra, tanto en memoria como en tiempo de ejecución (luego se tendrá que recorrer la estructura), que sólo tiene sentido pagar si el enunciado así lo requiere.

Por lo general, cuando se requiere almacenar, el enunciado dice "una vez leída y almacenada toda la información..." . Observar que, en estos casos, no se deben realizar otros procesamientos durante la lectura y almacenamiento (no sería correcto, por ejemplo, aprovechar para actualizar contadores y cuestiones así, no es lo que se espera).

2. Si se almacena toda la información, ¿se resuelven los incisos en un solo recorrido o en varios recorridos sobre la estructura de datos?

Por lo general, en la práctica se prioriza la eficiencia en tiempo de ejecución. O sea, se deben resolver los ejercicios con la menor cantidad de recorridos posibles sobre las estructuras de datos, diseñando módulos para atacar distintas partes del problema a un nivel más bajo (descomponer un número, evaluar una condición compleja, actualizar un vector, etc.).

Sin embargo, hay enunciados que dicen que, luego de almacenar toda la información, "*realice un módulo que haga eso, realice un módulo que haga aquello, etc.*". En estos casos, el enunciado está buscando que se modularice a un nivel más alto, para trabajar conceptos como reuso, independencia de módulos, etc. Aquí, se deberán hacer muchos recorridos.

Lo importante es saber que si se quiere ganar en modularización, posiblemente se pierda en eficiencia. Y que los enunciados dan las pistas necesarias para decidir qué camino tomar.

ALGUNOS EJEMPLOS

Ejemplo 1

Se dispone de un vector con números enteros, de dimensión física dimF y dimensión lógica dimL .

- Realizar un módulo que imprima el vector desde la primera posición hasta la última.
- Realizar un módulo que imprima el vector desde la última posición hasta la primera.
- Realizar un módulo que imprima el vector desde la mitad ($\text{dimL} \text{ DIV } 2$) hacia la primera posición, y desde la mitad más uno hacia la última posición.
- Realizar un módulo que reciba el vector, una posición X y otra posición Y , e imprima el vector desde la posición X hasta la Y . Asuma que tanto X como Y son menores o iguales a la dimensión lógica. Y considere que, dependiendo de los valores de X e Y , podría ser necesario recorrer hacia adelante o hacia atrás.
- Utilizando el módulo implementado en el inciso anterior, vuelva a realizar los incisos a , b y c .

Este ejercicio no pide que se cargue el vector, ya que el mismo se dispone. Se deberá declarar el tipo de datos para dicha estructura, e invocar a un módulo que cargue información en la estructura, pero no debe implementarse el módulo. Por otro lado, en cada inciso se piden módulos que requieren recorrer el

vector, con lo cual, si se invocan a los diferentes módulos desde el programa principal, se recorrerá el vector múltiples veces.

Ejemplo 2

La empresa Amazon Web Services (AWS) desea procesar información de sus 500 clientes monotributistas más grandes del país. De cada cliente conoce la fecha de firma del contrato con AWS, la categoría del monotributo (entre la A y la F), el código de la ciudad donde se encuentran las oficinas (entre 1 y 2400) y el monto mensual acordado en el contrato. La información se ingresa ordenada por fecha de firma de contrato (los más antiguos primero, los más recientes últimos).

Realizar un programa que lea y almacene la información de los clientes en una estructura de tipo vector. Una vez almacenados los datos, procesar dicha estructura para obtener:

- a. Cantidad de contratos por cada mes y cada año, y año en que se firmó la mayor cantidad de contratos.*
- b. Cantidad de clientes para cada categoría de monotributo*
- c. Código de las 10 ciudades con mayor cantidad de clientes*
- d. Cantidad de clientes que superan mensualmente el monto promedio entre todos los clientes.*

Este ejercicio requiere que su solución se organice en al menos dos partes: por un lado, se debe cargar el vector de clientes (esto debe realizarse en un módulo aparte que retorne el vector; dicho módulo podría utilizar otros módulos auxiliares, por ejemplo para leer un cliente por teclado). Por otro lado, se deberá recorrer el vector previamente cargado para resolver los incisos a..d en un único recorrido. Observar que el inciso d requerirá un recorrido adicional del vector, ya que durante el primer recorrido se debe calcular el monto promedio, y durante el segundo recorrido se deberán evaluar cuáles clientes superan dicho promedio.

Ejemplo 3

Realizar un programa que lea números enteros desde teclado hasta que se ingrese el valor -1 (que no debe procesarse) e informe:

- a. La cantidad de ocurrencias de cada dígito procesado.*
- b. El dígito más leído.*
- c. Los dígitos que no tuvieron ocurrencias.*

Este ejercicio requiere leer números y procesarlos para realizar los incisos a..c. En ningún momento se solicita almacenar los números leídos, con lo cual no debe generarse ninguna estructura que guarde todos los números.